¿Cómo Funciona y Cómo se Entrenó el Reconocedor HMM?

Guía Intuitiva y Paso a Paso para Entender el Proceso Estocástico

Universidad de los Llanos • Procesos Estocásticos • Autor: Camilo Mendoza

1. La Gran Pregunta y la Intuición Básica

¿Cómo puede una computadora saber qué letra estás dibujando en tiempo real si cada persona escribe diferente, con distinto tamaño y a distinta velocidad?

La analogía de la brújula: No importa si dibujas una letra gigante o diminuta, ni si tardas 1 segundo o 10 segundos. Si estás dibujando una **letra 'a'**, tu mano siempre hace el mismo viaje:

1. Empieza arriba a la derecha.
2. Curva hacia la izquierda y hacia abajo haciendo un círculo.
3. Sube a cerrar el círculo.
4. Baja verticalmente para hacer el vástago final.

El secreto del proyecto es **convertir ese viaje del lápiz en una serie de rumbos de brújula** y enseñarle a un modelo estocástico de probabilidades a reconocer ese camino.

2. El Rol del Dataset: La Materia Prima (data/)

El archivo `data/glifo-dataset-2026-11-06.json` contiene más de **5.500 trazos reales** hechos por personas. Para trabajar ordenadamente, lo dividimos en 27 archivos individuales dentro de `data/letras/` (como `a.json`, `b.json`, etc.).

Cada archivo guarda las coordenadas de pantalla puras:

`[ [x0, y0], [x1, y1], [x2, y2], ... ]`

Estos datos crudos son el "libro de texto" con el que la inteligencia artificial aprende. Sin estos datos, la máquina no sabría cómo luce una letra humana real.

3. La Transformación Mágica: De Dibujo a Símbolos

Los Modelos Ocultos de Markov (HMM) no entienden coordenadas continuas (como `x=142.5, y=320.1`). El módulo `core/FeatureExtractor.py` transforma ese dibujo mediante 3 pasos clave:

Paso 3.1: Invariancia al Tamaño (Normalización)
Calculamos la caja que rodea a la letra y la escalamos para que siempre mida `1.0 x 1.0`. Si la dibujas en una esquina pequeña o en toda la pantalla, el modelo la ve exactamente del mismo tamaño.

Paso 3.2: Invariancia a la Velocidad (Paso de Longitud de Arco)
Si escribes muy rápido, el ratón captura pocos puntos; si escribes lento, captura miles. Para solucionarlo, medimos la longitud de la línea y colocamos puntos virtuales equiespaciados cada `0.05` unidades. ¡Ahora la velocidad de escritura ya no afecta en absoluto!

Paso 3.3: La Brújula de 16 Sectores
Miramos la dirección entre cada par de puntos consecutivos y la convertimos en un número del 0 al 15:

- `0` = hacia el Este (derecha →)
- `4` = hacia el Norte (arriba ↑)
- `8` = hacia el Oeste (izquierda ←)
- `12` = hacia el Sur (abajo ↓)
- `16 al 19` = cuando levantas el lápiz para hacer un segundo trazo (ej. el

punto de la `i`).

Resultado: Tu dibujo de la `a` se convirtió en una secuencia de números como esta:

```
Secuencia de Observaciones O = [7, 8, 9, 10, 11, 12, 0, 1, 4, 12, 12]
```

4. ¿Qué es un HMM y la Topología Bakis?

Para cada letra del alfabeto construimos un **Modelo Oculto de Markov** λ = (A, B, π):

- **Estados Ocultos (S):** Son las etapas invisibles del trazo. Por ejemplo, en la 'a':
  Estado 0: iniciar el círculo • *Estado 1:* fondo de la curva • *Estado 2:* subir • *Estado 3:* bajar la pata.
- **Topología Bakis (Izquierda-Derecha):** Al escribir, el tiempo solo va hacia adelante. No puedes dibujar la pata final antes de hacer el círculo. La regla Bakis obliga a que desde el estado `S_i` solo se pueda quedar en `S_i`, avanzar a `S_i+1` o saltar a `S_i+2`. **Nunca se puede retroceder.**
- **Matriz A (Transición):** Probabilidad de pasar de una etapa a la siguiente.
- **Matriz B (Emisión):** Estando en cierta etapa, ¿qué dirección de brújula es más probable ver?

5. El Entrenamiento: Algoritmo de Baum-Welch (EM)

¿Cómo aprendió la computadora las matrices A y B para cada letra? Mediante el algoritmo de **Baum-Welch** (Esperanza-Maximización):

1. **Alineación Inicial:** Divide las secuencias de dibujo en tramos iguales para dar una primera suposición de las probabilidades.

2. **Paso E (Expectation):** Pasa las secuencias de entrenamiento usando las funciones *Forward* (α) y *Backward* (β) y calcula la probabilidad de qué estado produjo cada movimiento.
3. **Paso M (Maximization):** Ajusta las matrices A y B para aumentar la probabilidad de haber visto esos trazos reales.
4. **Repetición:** Se repite hasta 25 iteraciones hasta que las matrices se estabilizan y ya no cambian.

¿Por qué esto resolvió el problema entre 'a' y 'q'?
Anteriormente se usaban plantillas artificiales casi idénticas. Al entrenar con cientos de muestras humanas reales, Baum-Welch aprendió que la letra **'q'** emite una larga serie de símbolos hacia el sur profundo (`12, 12, 12`) con un vástago largo, mientras que la **'a'** hace una curva corta y se detiene en la base. ¡La confusión quedó resuelta al 100%!

6. ¿Qué hay en la carpeta models/? (El Cerebro Guardado)

El entrenamiento toma varios minutos. No podemos hacer esperar al usuario cada vez que abre el programa. Por eso, el resultado final de las matrices de probabilidad se guarda en los **54 archivos JSON de models/** (27 para 2D y 27 para 3D).

Cuando abres `main.py`, la aplicación carga esos JSON en **menos de 1 segundo** en memoria RAM. Están listos para predecir al instante.

7. En Tiempo Real: ¿Qué pasa cuando mueves el ratón?

Cada vez que mueves el ratón o rotas la esfera 3D:

1. El extractor calcula los símbolos del trazo parcial que llevas hasta ese milisegundo.
2. El algoritmo **Forward incremental** calcula en tiempo real qué tan probable es esa secuencia para cada uno de los 27 modelos: `P(letra | trazo_actual)`
3. El **RankingPanel** actualiza las barras de probabilidad en pantalla inmediatamente.
4. El algoritmo de **Viterbi** averigua en qué estado oculto vas y **proyecta con líneas punteadas el trazo futuro** que te falta por dibujar.

8. Resumen Comparativo de Términos

Concepto	¿Dónde está en el código?	¿Qué significa en lenguaje sencillo?
Dataset Maestro	`data/glifo-dataset-*.json`	Las 5.500 firmas y dibujos originales hechos por personas de carne y hueso.
Dataset por Letras	`data/letras/*.json`	Las muestras humanas organizadas en 27 carpetas/archivos para entrenar cada letra.
Extractor de Características	`core/FeatureExtractor.py`	El traductor que convierte trazos de ratón en direcciones de brújula (0..15).
Algoritmo Baum-Welch	`core/HMMEngine.py`	El profesor que ajusta las probabilidades de cada

		letra analizando los ejemplos.
Archivos de Modelos	`models/*.json`	Las matrices de probabilidad ya aprendidas y listas para usar.
Algoritmo Forward	`core/HMMEngine.py`	El motor en tiempo real que calcula qué porcentaje de parecido tiene tu trazo con cada letra.

Conclusión: El sistema es un puente elegante entre la geometría del trazo y la teoría estocástica de probabilidades, permitiendo clasificar y anticipar la intención humana en tiempo real.